

Design note — planar arm control

Review and adapt this draft in your own words before submission.

The system separates operator interaction, planning and execution. An `ament_python` controller owns state and motion; a `PyQt5/PyQtGraph` node sends typed service requests and visualizes published feedback. A small `ament_cmake` package generates the request and status types. All three joints share `JointState` topics: `joint_states` is feedback and `joint_commands` is reference telemetry.

The supplied kinematics file is unchanged. An adapter rejects nonfinite targets, nonplanar requests, below-ground targets, invalid IK angles and excessive Cartesian residual. It preserves an already-reached configuration. A quintic interpolates relative joint angles with zero endpoint velocity/acceleration; duration expands to satisfy reference speed and acceleration limits. Joint limits are preserved by monotonic interpolation. Interval bounds certify continuous ground clearance; an uncertifiable direct path is rejected. No general obstacle-routing claim is made.

The backend abstraction separates references from execution and measured state. Position mode is ideal kinematic playback. The optional velocity/PID mode includes velocity saturation, a bounded integral and an illustrative first-order velocity actuator, producing meaningful tracking error. It is not a dynamics/current model. The optional Gazebo backend bridges actual physical state and position targets. Completion checks measured position and velocity for 150 ms, with a settling timeout.

Pick/place is a controller-owned state machine with preflight checks and simulated grasp/release dwell (and physical attachment acknowledgements in Gazebo). Approach, lift, transfer and retreat surround the required operations. Busy requests are rejected; cancellation stops and holds, while success retains the settled target. Single moves may project beyond-reach targets, with requested/resolved coordinates shown in the GUI. Object operations reject projection to avoid claiming a grasp at the wrong point. Reset explicitly resets simulation, not physical hardware.

Execution/state publication runs at 50 Hz and status at 10 Hz. A two-thread ROS executor separates command planning from timer callbacks; shared state is locked. Qt widgets live on the main thread, with ROS callbacks communicating through queued signals and asynchronous service responses. Plot history is bounded; command/feedback timestamps are matched before calculating error. Stale telemetry disables new GUI commands. Measured timer jitter is reported; no hard-real-time guarantee is claimed. Motion-leg replanning still runs under the controller lock.

An orbitable solid-link workcell is a 3D rendering of the same planar FK, not a different robot. A separate recorder reuses an attributed Apache-2.0 CSV/JSONL helper; it observes state/reference/status without commanding motion or adding disk I/O to the controller. Other reviewed robots' IK and meshes are not reused.

Validation combines independent geometry tests, a separate-process ROS witness and a reproducible GUI recording. The witness checks constraints, sequence order, endpoint accuracy, projection, rejection, cancellation and message timing. The library hash is checked against the supplied archive. Known library limitations include unconstrained fallback IK, origin-target early return and an unused beta correction in the analytical branch; these are documented rather than edited.

With more time, priorities are a standard pick/place action, worker-based planning for every transition, a richer path search, broader fault tests and a hardware backend with measured feedback and device-side watchdogs. Gazebo uses a simplified attachment grasp and fixture collisions, with a documented 5 mm release gap; it is not frictional-grasp or hardware validation. Current control is not implemented.